Clustering and principal component analysis

A focused six-page guide: groups, geometry, and compression

1. Clustering and the K-means objective

Our data matrix $X \in \mathbb{R}^{n \times p}$ has one observation per row and one feature per column. There is no target label to predict. **Clustering** assigns group identifiers; **principal component analysis (PCA)** constructs numerical coordinates. Neither automatically discovers meaningful categories. Before fitting, state what a row represents and why two rows should count as similar.

Distance encodes a choice. Euclidean squared distance is $\|x - z\|^2 = \sum_{j=1}^{p} (x_j - z_j)^2$. Multiplying one feature by 100 multiplies its contribution by 10,000. For comparable weighting, we may standardize $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$, with $s_j > 0$ a chosen scale. This replaces distance by $\sum_j (x_{ij} - x_{\ell j})^2 / s_j^2$; it changes the question being asked. Constant columns need removal or a documented convention. State names and row IDs are not numerical features.

K-means. Choose K nonempty groups $C_1, \ldots, C_K$ with centers $\mu_1, \ldots, \mu_K$. The within-cluster sum of squared distances, also called *inertia*, is

$$\boxed{W = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2.}$$

For fixed groups, minimizing each quadratic gives $\mu_k = |C_k|^{-1} \sum_{i \in C_k} x_i$. For fixed centers, assign each row to its nearest center. **Lloyd's algorithm** alternates these two steps. Each exact step cannot increase W, but the joint problem is not convex: different starting centers may give different answers. Resolve distance ties consistently; an empty group needs an explicit policy.

A hand calculation. Start with $x = (0, 2, 3, 9, 10, 12)$, $K = 2$, centers $(0, 3)$. The nearest-center boundary is 1.5, so groups are $\{0\}$ and $\{2, 3, 9, 10, 12\}$. Updating gives centers $(0, 7.2)$ and $W = 78.8$. The next boundary is 3.6, producing $\{0, 2, 3\}$ and $\{9, 10, 12\}$. Their means are $5/3$ and $31/3$, with $W = 28/3$. Assignments then stay fixed. Initial W was 167: the objective has decreased at every effective update.

Practical Python. `KMeans(n_clusters=3, n_init=20, random_state=23)` runs 20 initializations and keeps the lowest-inertia result. `fit(X)` learns centers, `labels_` stores fitting-row assignments, `inertia_` stores W, and `predict(X_new)` assigns new rows to the learned centers. It returns cluster IDs, not supervised class predictions. IDs may be permuted between equivalent fits.

Remember. K-means creates convex nearest-center regions and favors compact groups. Two concentric rings are a useful counterexample: low squared-distance cost need not recover ring membership.

2. Hierarchies and cluster assessment

Agglomerative clustering starts with one group per row and repeatedly merges two groups. A **dendrogram** records merges; a horizontal cut chooses groups. Horizontal leaf order is arbitrary, and vertical height must be interpreted using the linkage definition. For groups A and B and Euclidean distance d:

Linkage	Criterion minimized at each merge
Single	$\min_{i\in A, j\in B} d(x_i, x_j)$
Complete	$\max_{i\in A, j\in B} d(x_i, x_j)$
Average	$(\lvert A\rvert\lvert B\rvert)^{-1} \sum_{i\in A, j\in B} d(x_i, x_j)$
Ward	$\Delta(A, B) = \frac{\lvert A\rvert\lvert B\rvert}{\lvert A\rvert + \lvert B\rvert} \lVert \bar{x}_A - \bar{x}_B \rVert^2$

Ward minimizes the increase in within-group squared error and requires Euclidean geometry. Single linkage can trace nonconvex groups, but a thin bridge can join otherwise separated clouds. Complete linkage discourages large diameters; no linkage is universally best.

For points $0, 1, 4, 6$, all four methods first form $\{0, 1\}$ and $\{4, 6\}$. Their last single, complete, and average heights are $3, 6, 4.5$. Ward's last cost is 20.25. SciPy reports Ward height $\sqrt{2\Delta}$, here $\sqrt{40.5} \approx 6.364$, not the raw cost. Keep distance units separate from squared-error units. `AgglomerativeClustering(...).fit_predict(X)` assigns fitting rows; it has no standard `predict` method for new rows. A new-point assignment rule would be an extra choice.

Choosing K. Globally optimal inertia cannot increase as K grows, so minimizing inertia alone selects too many groups. An elbow is a diagnostic, not a proof. Local fits may even violate the ideal monotonic pattern. For row i, let a_i be its average distance to other members of its own group and b_i the smallest average distance to any other group. The **silhouette** is

$$\boxed{s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \qquad -1 \le s_i \le 1.}$$

High values indicate compact, separated groups in the chosen metric. Negative values suggest another group is closer on average. Singletons receive zero in the software convention. The mean requires between 2 and $n - 1$ distinct groups. It can prefer compact cuts to meaningful curved groups.

Compare partitions without matching IDs. The **adjusted Rand index (ARI)** compares pairwise same-group decisions, corrects chance agreement under its reference model, equals 1 for identical partitions, and can be negative. Relabeling IDs leaves it unchanged. Generating labels in a simulation are for a post-fit audit, not unsupervised selection. Stability means comparing refitted partitions on shared rows across subsamples; it does not prove the groups are useful or real.

Computational scale. A Lloyd assignment step costs roughly $O(nKp)$ operations. Dense pairwise distances need $O(n^2)$ storage; hierarchies and exact silhouette calculations can become expensive before K-means does. For larger datasets, a documented silhouette subsample or a mini-batch clustering method can be practical, but changes the computation being reported.

3. PCA: directions, scores, and explained variance

Let $\bar{x}$ be the vector of fitting-column means and $Z = X - \mathbf{1}\bar{x}^T$ the centered data. Its sample covariance matrix is $S = Z^T Z/(n-1)$. For a unit vector v, projected scores are Zv and their sample variance is $v^T Sv$. The first **principal component (PC)** direction solves

$$\boxed{\max_{\|v\|=1} v^T Sv.}$$

A Lagrange multiplier gives $Sv = \lambda v$. Thus choose a leading eigenvector v_1; its eigenvalue λ_1 is the score variance. Later directions maximize remaining variance subject to orthogonality. Sort $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$ and collect q directions in $V_q = [v_1, \ldots, v_q] \in \mathbb{R}^{p\times q}$.

Direction versus score. A direction has p entries describing a linear combination of features; a score is the coordinate of one observation along it. The score matrix is

$$\boxed{T_q = ZV_q \in \mathbb{R}^{n\times q}, \qquad t_{ij} = (x_i - \bar{x})^T v_j.}$$

The word *loading* is used for either direction coefficients or scaled quantities in different sources. Our tables report direction coefficients explicitly. A component's sign is arbitrary: changing v_j to $-v_j$ changes the score sign but preserves the subspace, distances, and reconstructions. Repeated eigenvalues also allow different orthonormal bases within the same eigenspace.

Explained variance. When total variance is positive,

$$\mathrm{PVE}_j = \frac{\lambda_j}{\sum_\ell \lambda_\ell}, \qquad \mathrm{CPVE}(q) = \frac{\sum_{j=1}^q \lambda_j}{\sum_\ell \lambda_\ell}.$$

PVE means proportion of variance explained; CPVE is its cumulative sum. Keeping 90% variance is a compression convention, not a guarantee of 90% classification accuracy or of retaining every important group. A small-variance direction may contain the information needed for another task.

Centering is not scaling. PCA centers automatically but does not standardize feature variances. With mixed units, compare centered PCA to PCA after an explicit scaler. `StandardScaler` uses fitting variance with divisor n; our covariance eigenvalues use $n-1$. For complete data with nonconstant features, using sample instead of population standard deviations multiplies all standardized columns by the same $\sqrt{(n-1)/n}$: directions and PVE agree, absolute eigenvalues differ. A larger column mean alone does not dominate a correctly centered PCA.

Remember. PCA seeks variation, without knowing what is signal, noise, a cluster, or a target. Choosing the input geometry is part of the analysis, not a harmless preparation step.

4. SVD, reconstruction, and a complete example

The thin **singular value decomposition (SVD)** is $Z = UDV^T$, with descending singular values $d_j \geq 0$ on D. Orthonormal columns of V are PCA directions, and

$$\boxed{\lambda_j = \frac{d_j^2}{n-1}, \qquad T_q = U_q D_q = ZV_q.}$$

Centering implies rank at most $\min(n-1, p)$; thin SVD may still return a zero singular value. Working directly with Z avoids explicitly forming $Z^T Z$, which squares its condition number.

Project, then reconstruct. The rank-q reconstruction in original coordinates is

$$\boxed{\widehat{X}_q = \mathbf{1}\bar{x}^T + T_q V_q^T.}$$

Among all orthogonal projections onto q-dimensional linear subspaces of centered feature space, PCA minimizes squared reconstruction error. Pythagoras decomposes retained and discarded parts, giving

$$\|Z - ZV_qV_q^T\|_F^2 = \sum_{j>q} d_j^2 = (n-1)\sum_{j>q} \lambda_j.$$

The squared Frobenius norm $\|A\|_F^2$ is the sum of squares of all entries. Dividing by np gives **mean squared error (MSE) per entry**; dividing by n instead gives squared error per row. State the denominator before comparing numbers.

Exact example. These four rows are already centered:

$$Z = \begin{pmatrix} 3 & 1 \\ 1 & 3 \\ -1 & -3 \\ -3 & -1 \end{pmatrix}, \quad S = \tfrac{1}{3}\begin{pmatrix} 20 & 12 \\ 12 & 20 \end{pmatrix}, \quad v_1 = \tfrac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad v_2 = \tfrac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Eigenvalues are $32/3$ and $8/3$, so PC1 explains 80%. Its scores are $(2\sqrt{2}, 2\sqrt{2}, -2\sqrt{2}, -2\sqrt{2})$. With $q = 1$, reconstructed rows are $(2,2), (2,2), (-2,-2), (-2,-2)$. Each residual has squared length 2: total error is 8 and per-entry MSE is $8/(4 \cdot 2) = 1$. This agrees with the discarded-eigenvalue identity.

Python mapping. `PCA(n_components=q, svd_solver="full")` specifies a dense full SVD. `fit(X_train)` learns `mean_` and directions stored as *rows* of `components_`. `transform(X_new)` centers using that same mean and returns scores. `inverse_transform(scores)` reconstructs p features. `explained_variance_` stores eigenvalues and `explained_variance_ratio_` stores PVE. For standardized PCA, invert PCA first and the scaler second to return to original units.

Remember. Use the fitting mean, scale, and directions for new rows. Calling `fit_transform` again on test rows learns a different coordinate system and no longer assesses the frozen representation.

5. Compression, denoising, and clustering after PCA

Compression and denoising ask different questions. Suppose $x = s + \epsilon$ contains a clean signal s and noise ϵ. Reconstruction error to observed x decreases as more components are retained; with a full p-dimensional basis it is zero. This alone cannot decide how much noise should be removed. Error to s may first decrease and then increase as noise directions are reintroduced. A clean validation target can select a denoising dimension, but such a target is usually unavailable. Our simulation labels this comparison an *oracle diagnostic* rather than a deployable selection method.

With a known mean, isotropic noise variance σ^2, and a fixed q-dimensional orthogonal projection P independent of that noise,

$$\mathbb{E}\|Px - s\|^2 = \|(I - P)s\|^2 + q\sigma^2.$$

The first term is discarded signal; the second is retained noise. Estimated means and subspaces introduce additional effects, so this is intuition rather than an exact formula for every fitted experiment. A large variance cutoff can keep many noise directions if the noise dimension is large.

A full rotation preserves distance. For a p-dimensional orthogonal PCA basis V, $\|(x-z)^T V\|^2 = \|x - z\|^2$. K-means has the same objective geometry before and after this full rotation (provided scaling is unchanged). Truncating to q directions instead gives

$$\|x - z\|^2 = \|(x - z)^T V_q\|^2 + \|(x - z)^T V_\perp\|^2.$$

Two points may become indistinguishable in a two-PC plot while differing in discarded coordinates. Never assume clustering all features must match clustering the plotted scores. Whitening additionally divides each score by $\sqrt{\lambda_j}$; it changes distances and can amplify low-variance noise. The notebooks leave whitening off.

Match the evaluation to the goal.

- A full-data clustering or PCA description summarizes the observed rows. It has no test accuracy.
- For new-row compression, split before estimating means and directions. Freeze a component rule on fitting data; use validation only for any declared selection, and evaluate the final policy on test data.
- For stability, subsample rows, refit all preprocessing and clustering, and compare assignments on overlapping rows. Do not compare raw cluster numbers or describe pairwise stability scores as independent replicates.
- For downstream supervised prediction, include the scaler and PCA inside each training cross-validation fold. Unsupervised preprocessing can still leak evaluation information.

Storage intuition. A dense compressed dataset stores nq scores, pq direction entries, and p means, versus np original entries. New-row scoring costs roughly pq multiply-add terms per row. A two-dimensional plot is useful communication, but its visual separation is not validation of a clustering.

6. Applications, student exercises, and useful checks

A: cluster shape and stability. Verify the six-point Lloyd calculation and four-point merge heights. Generate three Gaussian clouds; select K from 2 to 6 by maximum silhouette, then audit generating-label ARI. Compare K-means and four hierarchies on clouds and rings. Repeat fixed-K clustering on subsamples to separate sample sensitivity from initialization sensitivity. The analysis is descriptive, with no artificial classification target.

B: PCA compression and denoising. Verify the exact four-row example using SVD and scikit-learn. Generate 600 rows, 20 features, rank-three signal plus isotropic noise. Use a 60/20/20 split; compare fixed $q = 3$, the smallest fitting q reaching 90% variance, and a clearly marked clean-validation oracle. Report test errors against both noisy observations and clean signal.

C: USArrests. Download 50 historical state records directly from a small CSV. Compare centered and standardized PCA, interpret direction coefficients, reconstruct two-PC approximations, and inspect K-means/Ward groups. Refit scaling in subsampling diagnostics. The data describe historical aggregates; groups are neither current state ratings nor individual risk estimates.

D: student worksheet. Two generated-data exercises: fit K-means and assign new rows; fit PCA, compute scores, and reconstruct held-out rows. Five short code gaps have explicit comments. Separate PDF solutions explain the answers, implementation choices, and checks.

Check your understanding.

1. For points 1, 2, 6 in one cluster, compute its mean and inertia. Why does the mean minimize inertia?
2. For $a_i = 2$, $b_i = 5$, compute silhouette. Can high average silhouette prove the chosen groups have semantic meaning?
3. A centered matrix has singular values 6 and 2 with $n = 5$. Find the first PVE, eigenvalues, and rank-one total error.
4. Two runs return labels $(0, 0, 1, 1)$ and $(7, 7, 3, 3)$. Do they disagree? What does ARI equal?
5. If PCA retains all p directions without whitening, can a pure rotation change exact Euclidean distances? What changes after truncation?

Answer checkpoints. (1) mean 3, inertia 14; differentiating squared error gives zero at the mean. (2) 3/5; no. (3) PVE $36/40 = 0.9$, eigenvalues 9 and 1, error 4. (4) same partition, ARI 1. (5) no; truncation removes discarded-direction squared distances.

Before trusting a result: check feature units, missing values, row identity, random seeds, restart count, and whether a plot uses the same space as a reported metric. Distinguish raw inertia, per-row error, and per-entry error. Check projections rather than eigenvector signs. Explain what selected K or q, and keep external labels out of unsupervised selection.

Documentation. See the official KMeans, PCA, linkage, and USArrests references. Notebooks use the course's scikit-learn 1.8 environment. The longer lecture gives further derivations and extensions.